

Module 0: Python Installation Guide

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Install Python on Windows, macOS, and Linux
- Set up a development environment
- Install required packages for the course
- Verify your installation
- Troubleshoot common installation issues

Prerequisites

- Basic computer skills
- Internet connection
- Administrator/sudo privileges (for some installations)

Overview

This module will guide you through installing Python and setting up your development environment for this course. We'll cover multiple installation methods and provide platform-specific instructions.

Table of Contents

1. [System Requirements](#)
2. [Python Installation Methods](#)
3. [Platform-Specific Instructions](#)
4. [Package Installation](#)
5. [Verification](#)
6. [Troubleshooting](#)
7. [Next Steps](#)

1. System Requirements

Minimum Requirements

- **Operating System:** Windows 10+, macOS 10.14+, or Linux (Ubuntu 18.04+, CentOS 7+, etc.)

- **RAM:** 4GB minimum, 8GB recommended
- **Storage:** 2GB free space
- **Internet Connection:** Required for downloading packages

Recommended Requirements

- **RAM:** 8GB or more
- **Storage:** 10GB free space
- **CPU:** Multi-core processor
- **Graphics:** Dedicated graphics card (optional, for visualization)

Python Version

- **Required:** Python 3.8 or higher
- **Recommended:** Python 3.9, 3.10, or 3.11
- **Not Supported:** Python 2.x (end of life)

Check Your Current Python Version

If you already have Python installed, check your version:

```
In [ ]: # Check Python version
import sys
print(f"Python version: {sys.version}")
print(f"Python executable: {sys.executable}")
```

```
In [ ]: # Run this in your terminal/command prompt to check Python version
# python --version
# or
# python3 --version

# If you're running this in Jupyter, you can check with:
import sys
print(f"Python version: {sys.version}")
print(f"Python executable: {sys.executable}")
```

2. Python Installation Methods

Method 1: Official Python Installer (Recommended for Beginners)

Pros: Simple, official, includes pip **Cons:** May not be the latest version immediately

Method 2: Package Managers

Pros: Easy updates, dependency management **Cons:** Platform-specific

- **Windows:** Chocolatey, Scoop
- **macOS:** Homebrew, MacPorts
- **Linux:** apt, yum, pacman

Method 3: Anaconda/Miniconda (Recommended for Data Science)

Pros: Includes data science packages, environment management **Cons:** Larger download, may include packages you don't need

Method 4: pyenv (Advanced Users)

Pros: Multiple Python versions, project-specific environments **Cons:** More complex setup

3. Platform-Specific Instructions

Windows Installation

Option A: Official Python Installer

1. Download Python:

- Go to python.org
- Click "Download Python 3.x.x"
- Choose the latest stable version

2. Run the Installer:

- Double-click the downloaded `.exe` file
- **IMPORTANT:** Check "Add Python to PATH" at the bottom
- Choose "Install Now" or "Customize installation"
- If customizing, ensure pip is selected

3. Verify Installation:

- Open Command Prompt (cmd) or PowerShell
- Type: `python --version`
- Type: `pip --version`

Option B: Using Chocolatey

1. Install Chocolatey (if not already installed):

```
# Run as Administrator
Set-ExecutionPolicy Bypass -Scope Process -Force
[System.Net.ServicePointManager]::SecurityProtocol =
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072
iex ((New-Object
System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))
```

2. Install Python:

```
choco install python
```

Option C: Using Scoop

1. Install Scoop (if not already installed):

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser  
irm get.scoop.sh | iex
```

2. Install Python:

```
scoop install python
```

macOS Installation

Option A: Official Python Installer

1. Download Python:

- Go to python.org
- Click "Download Python 3.x.x"
- Download the macOS installer (.pkg file)

2. Install Python:

- Double-click the downloaded `.pkg` file
- Follow the installation wizard
- Python will be installed in `/Applications/Python 3.x/`

3. Verify Installation:

- Open Terminal
- Type: `python3 --version`
- Type: `pip3 --version`

Option B: Using Homebrew (Recommended)

1. Install Homebrew (if not already installed):

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install Python:

```
brew install python
```

3. Verify Installation:

```
python3 --version  
pip3 --version
```

Option C: Using MacPorts

1. Install MacPorts:

- Download from macports.org
- Install the package

2. Install Python:

```
sudo port install python311
```

Linux Installation

Ubuntu/Debian

1. Update package list:

```
sudo apt update
```

2. Install Python:

```
sudo apt install python3 python3-pip python3-venv
```

3. Install development tools (optional but recommended):

```
sudo apt install python3-dev build-essential
```

CentOS/RHEL/Fedora

For CentOS/RHEL:

```
sudo yum install python3 python3-pip
```

or for newer versions

```
sudo dnf install python3 python3-pip
```

For Fedora:

```
sudo dnf install python3 python3-pip
```

Arch Linux

```
sudo pacman -S python python-pip
```

OpenSUSE

```
sudo zypper install python3 python3-pip
```

Anaconda/Miniconda Installation (Cross-Platform)

Anaconda (Full Distribution)

1. Download Anaconda:

- Go to anaconda.com
- Download the installer for your platform

2. Install Anaconda:

- **Windows:** Run the `.exe` installer
- **macOS:** Run the `.pkg` installer
- **Linux:** Run the `.sh` installer
- Follow the installation wizard

3. Verify Installation:

```
conda --version  
python --version
```

Miniconda (Minimal Distribution)

1. Download Miniconda:

- Go to docs.conda.io

- Download the installer for your platform

2. Install Miniconda:

- Same process as Anaconda
- Choose "Add to PATH" when prompted

3. Create Environment:

```
conda create -n python-course python=3.11
conda activate python-course
```

4. Package Installation

Virtual Environments (Recommended)

Virtual environments help isolate your project dependencies and prevent conflicts.

Using venv (Built-in)

1. Create virtual environment:

```
python -m venv python-course-env
```

2. Activate virtual environment:

```
# Windows
python-course-env\Scripts\activate
```

```
# macOS/Linux
source python-course-env/bin/activate
```

3. Deactivate when done:

```
deactivate
```

Using conda

1. Create conda environment:

```
conda create -n python-course python=3.11
```

2. Activate environment:

```
conda activate python-course
```

Installing Required Packages

Method 1: Using pip with requirements.txt

1. Navigate to course directory:

```
cd /path/to/python-course
```

2. Install packages:

```
pip install -r requirements.txt
```

Method 2: Using conda

1. Install from environment.yml:

```
conda env create -f environment.yml
conda activate python-course
```

2. Or install manually:

```
conda install numpy pandas scipy scikit-learn matplotlib
seaborn jupyter
```

Method 3: Install packages individually

Core packages

```
pip install numpy pandas scipy
```

Machine learning

```
pip install scikit-learn
```

Visualization

```
pip install matplotlib seaborn
```

Jupyter

```
pip install jupyter jupyterlab
```

Optional packages

```
pip install plotly bokeh xgboost lightgbm
```

Package Versions

The course requires the following minimum versions:

Package	Minimum Version	Purpose
numpy	1.21.0	Numerical computing
pandas	1.3.0	Data manipulation
scipy	1.7.0	Scientific computing
scikit-learn	1.0.0	Machine learning
matplotlib	3.5.0	Basic plotting
seaborn	0.11.0	Statistical visualization
jupyter	1.0.0	Interactive computing

5. Verification

After installation, verify your setup by running the following code:

```
In [ ]: # Test basic imports
try:
    import numpy as np
    print(f"✅ NumPy {np.__version__} - OK")
except ImportError:
    print("❌ NumPy - FAILED")
```

```

try:
    import pandas as pd
    print(f"✅ Pandas {pd.__version__} - OK")
except ImportError:
    print("❌ Pandas - FAILED")

try:
    import matplotlib.pyplot as plt
    print(f"✅ Matplotlib {plt.matplotlib.__version__} - OK")
except ImportError:
    print("❌ Matplotlib - FAILED")

try:
    import seaborn as sns
    print(f"✅ Seaborn {sns.__version__} - OK")
except ImportError:
    print("❌ Seaborn - FAILED")

try:
    from sklearn import datasets
    import sklearn
    print(f"✅ Scikit-learn {sklearn.__version__} - OK")
except ImportError:
    print("❌ Scikit-learn - FAILED")

try:
    import scipy.stats
    import scipy
    print(f"✅ SciPy {scipy.__version__} - OK")
except ImportError:
    print("❌ SciPy - FAILED")

print("\n" + "="*50)
print("INSTALLATION VERIFICATION COMPLETE")
print("="*50)

```

```

In [ ]: # Test basic functionality
print("Testing basic functionality...")
print("\n1. NumPy array creation:")
arr = np.array([1, 2, 3, 4, 5])
print(f"    Array: {arr}")
print(f"    Mean: {np.mean(arr)}")

print("\n2. Pandas DataFrame creation:")
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
print(f"    DataFrame:\n{df}")

print("\n3. Matplotlib plot (basic):")
plt.figure(figsize=(6, 4))
plt.plot([1, 2, 3, 4], [1, 4, 2, 3])
plt.title('Test Plot')
plt.show()

print("\n4. Scikit-learn dataset:")
iris = datasets.load_iris()
print(f"    Iris dataset shape: {iris.data.shape}")
print(f"    Target names: {iris.target_names}")

```



```
print("\n✅ All basic functionality tests passed!")
print("Your Python environment is ready for the course.")
```

6. Troubleshooting

Common Issues and Solutions

Issue 1: "python: command not found" (Linux/macOS)

Problem: Python is not in your PATH

Solutions:

1. Try `python3` instead of `python`
2. Add Python to your PATH:
`export PATH="/usr/local/bin/python3:$PATH"`
3. Create an alias:
`alias python=python3`
`alias pip=pip3`

Issue 2: "pip: command not found"

Problem: pip is not installed or not in PATH

Solutions:

1. Install pip:
Linux
`sudo apt install python3-pip`

macOS (with Homebrew)
`brew install python`
2. Use `python -m pip`:
`python -m pip install package_name`

Issue 3: "Permission denied" errors

Problem: Installing packages system-wide requires admin privileges

Solutions:

1. Use a virtual environment (recommended)
2. Use `--user` flag:
`pip install --user package_name`
3. Use `sudo` (not recommended):
`sudo pip install package_name`

Issue 4: ImportError when running Jupyter

Problem: Jupyter is using a different Python environment

Solutions:

1. Install Jupyter in the same environment:
`pip install jupyter`
2. Install ipykernel:
`pip install ipykernel`
`python -m ipykernel install --user --name=python-course`
3. Select the correct kernel in Jupyter

Issue 5: Matplotlib backend issues

Problem: Plots not displaying or GUI issues

Solutions:

1. Install GUI backend:
`pip install matplotlib[all]`
2. Set backend:
`import matplotlib`
`matplotlib.use('Agg') # For headless servers`
3. Install system dependencies:
Ubuntu/Debian
`sudo apt install python3-tk`

macOS (with Homebrew)
`brew install python-tk`

Issue 6: Package version conflicts

Problem: Different packages require different versions of the same dependency

Solutions:

1. Use virtual environments
2. Use conda for better dependency resolution
3. Update all packages:
`pip install --upgrade pip setuptools wheel`
`pip install --upgrade package_name`

Platform-Specific Issues

Windows

- **Issue:** Long path names
 - **Solution:** Enable long paths in Windows 10
- **Issue:** Antivirus blocking installations
 - **Solution:** Add Python directories to antivirus exceptions

macOS

- **Issue:** Xcode command line tools missing
 - **Solution:** `xcode-select --install`
- **Issue:** Homebrew permission issues
 - **Solution:** `sudo chown -R $(whoami) /usr/local/bin`

Linux

- **Issue:** Missing system dependencies
 - **Solution:** Install build essentials
`sudo apt install build-essential python3-dev`
- **Issue:** SSL certificate errors
 - **Solution:** Update certificates
`sudo apt install ca-certificates`

7. Next Steps

After Successful Installation

1. Explore Jupyter:

- Start Jupyter: `jupyter notebook` or `jupyter lab`
- Create a new notebook
- Try running some basic Python code

2. Set up your development environment:

- Choose a code editor (VS Code, PyCharm, etc.)
- Configure Python interpreter
- Install useful extensions

3. Practice with the basics:

- Run through Python basics
- Try importing and using the installed packages
- Create simple data analysis scripts

Recommended Development Setup

Code Editors

1. Visual Studio Code (Free, recommended)

- Install Python extension
- Install Jupyter extension
- Configure Python interpreter

2. PyCharm (Professional IDE)

- Community Edition (free)
- Professional Edition (paid)
- Built-in Jupyter support

3. Spyder (Scientific IDE)

- Comes with Anaconda
- Great for data science
- Built-in variable explorer

Useful Extensions and Tools

1. Jupyter Extensions:

- JupyterLab extensions
- Jupyter notebook extensions
- Variable inspector
- Code formatter

2. Python Tools:

- Black (code formatter)
- Flake8 (linter)
- pytest (testing)
- ipython (enhanced REPL)

Getting Help

If you encounter issues:

1. Check documentation:

- [Python.org](https://python.org)
- [NumPy docs](https://numpy.org/doc/)
- [Pandas docs](https://pandas.pydata.org/pandas-docs/)
- [Scikit-learn docs](https://scikit-learn.org/)

2. Community resources:

- [Stack Overflow](https://stackoverflow.com/)
- [Python Reddit](https://www.reddit.com/r/Python/)
- [Data Science Stack Exchange](https://data.stackexchange.com/)

3. Course-specific help:

- Check the course discussion forum
- Contact the instructor
- Review the troubleshooting section above

Environment Management Best Practices

1. **Always use virtual environments** for projects
2. **Keep requirements files updated**
3. **Document your environment** setup
4. **Regularly update packages** (but test first)
5. **Use version control** (git) for your code

Final Checklist

Before starting Module 1, ensure you have:

- ☐ Python 3.8+ installed and working

- ☐ pip installed and working
- ☐ Virtual environment created and activated
- ☐ All required packages installed
- ☐ Jupyter notebook/lab working
- ☐ Verification tests passing
- ☐ Code editor configured
- ☐ Basic understanding of command line

Congratulations!

You've successfully set up your Python development environment! You're now ready to begin the Python course. Start with Module 1: Introduction and Essentials of Python.

Quick Start Commands

```
# Activate your environment
source python-course-env/bin/activate # Linux/macOS
# or
python-course-env\Scripts\activate    # Windows

# Start Jupyter
jupyter notebook
# or
jupyter lab

# Deactivate when done
deactivate
Happy coding!
```